

Documentation Complète

181 Endpoints · Requêtes & Réponses · Backend + Frontend

Ce document est la référence officielle pour le développeur Backend (P3/P4) et les développeurs Frontend (P1/P2). Chaque endpoint est documenté avec sa requête complète, ses paramètres, sa réponse JSON et des notes spécifiques pour chaque partie.

VERSION	ENDPOINTS	BASE URL	AUTH
v1.0	181	/api/v1	Bearer JWT

Toutes les requêtes authentifiées doivent inclure le header : Authorization: Bearer –
Content-Type: application/json

Table des Matières

01	Auth & Utilisateurs	14 endpoints
02	Organisations & Workspaces	14 endpoints
03	Pipelines — CRUD	19 endpoints
04	Nœuds & Edges	14 endpoints
05	Exécution & Runs	16 endpoints
06	Fichiers & Datasources	16 endpoints
07	Transformations SQL	9 endpoints
08	Intelligence Artificielle	13 endpoints
09	Résultats & Exports	10 endpoints
10	Scheduling	9 endpoints
11	Webhooks	9 endpoints
12	Notifications & Alertes	10 endpoints
13	Analytics & Audit	6 endpoints
14	API Keys & Intégrations	8 endpoints
15	Santé & Ops	6 endpoints
16	Endpoints Avancés (n8n+)	8 endpoints

Auth & Utilisateurs

Inscription, connexion, sessions et profil

POST /api/v1/auth/register

Créer un compte

Enregistre un nouvel utilisateur. Envoie un email de vérification après création.

Corps de la requête (JSON)

Champ	Type		Description
email	string	● req	Email valide — utilisé pour la connexion
password	string	● req	Min 8 caractères, 1 majuscule, 1 chiffre
name	string	● req	Nom complet affiché dans l'interface
org_name	string	■ opt	Nom de l'organisation à créer automatiquement

Exemple de réponse

```
{ "user": { "id": "usr_01J...", "email": "john@acme.com",  
  "name": "John Doe", "verified": false },  
  "message": "Verification email sent" }
```

■ Le mot de passe est hashé avec bcrypt (cost 12) avant stockage.

■ Frontend : Rediriger vers /verify-email après succès. Afficher le message de vérification.

■ Backend : Générer un token JWT de vérification (expire dans 24h). Hash password avant INSERT.

POST /api/v1/auth/login

Se connecter

Authentifie un utilisateur. Retourne un access_token (15min) et un refresh_token (30j).

Corps de la requête (JSON)

Champ	Type		Description
email	string	● req	Email du compte
password	string	● req	Mot de passe

Exemple de réponse

```
{ "access_token": "eyJhbGciOiJIUzI1NiJ9...",  
  "refresh_token": "eyJhbGciOiJIUzI1NiJ9...",  
  "expires_in": 900,  
  "user": { "id": "usr_01J...", "name": "John Doe", "org_id": "org_01J..." } }
```

■ Frontend : Stocker access_token en mémoire (pas localStorage). refresh_token en httpOnly cookie.

■ Backend : Créer une entrée session en DB avec user_agent et ip. Limiter à 5 tentatives/min (rate limit).

POST /api/v1/auth/refresh-token

Rafraîchir le token d'accès

Échange un refresh_token valide contre un nouvel access_token.

Corps de la requête (JSON)

Champ	Type		Description
-------	------	--	-------------

Champ	Type		Description
refresh_token	string	● req	Token de rafraîchissement obtenu au login

Exemple de réponse

```
{ "access_token": "eyJhbGciOiJIUzI1NiJ9...", "expires_in": 900 }
```

- Frontend : Appeler automatiquement quand une requête retourne 401. Intercepteur Axios recommandé.
- Backend : Invalider le refresh_token utilisé (rotation). Vérifier qu'il n'est pas blacklisté.

POST /api/v1/auth/logout

Se déconnecter

Invalide la session courante. Le refresh_token est blacklisté.

Exemple de réponse

```
{ "message": "Logged out successfully" }
```

- Inclure le refresh_token dans le body pour l'invalider côté serveur.
- Frontend : Vider le state global (user, token). Rediriger vers /login.
- Backend : Blacklister le refresh_token en Redis avec TTL = durée de vie restante.

GET /api/v1/auth/me

Profil de l'utilisateur connecté

Retourne les informations complètes de l'utilisateur authentifié.

Exemple de réponse

```
{ "id": "usr_01J...", "email": "john@acme.com", "name": "John Doe",  
  "verified": true, "created_at": "2026-06-01T10:00:00Z",  
  "orgs": [{ "id": "org_01J...", "name": "Acme Corp", "role": "owner" }] }
```

- Frontend : Appeler au montage de l'app pour initialiser le state utilisateur.

PATCH /api/v1/auth/me

Modifier son profil

Met à jour le nom ou l'avatar de l'utilisateur connecté.

Corps de la requête (JSON)

Champ	Type		Description
name	string	■ opt	Nouveau nom affiché
avatar_url	string	■ opt	URL d'un avatar externe

Exemple de réponse

```
{ "id": "usr_01J...", "name": "John Updated", "avatar_url": "..." }
```

POST /api/v1/auth/change-password

Changer le mot de passe

Modifie le mot de passe en vérifiant l'ancien.

Corps de la requête (JSON)

Champ	Type		Description
current_password	string	● req	Mot de passe actuel

new_password	string	● req	Nouveau mot de passe (min 8 cars)
---------------------	--------	-------	-----------------------------------

Exemple de réponse

```
{ "message": "Password updated. All sessions revoked." }
```

■ Backend : Invalider toutes les sessions actives après changement de mot de passe.

POST /api/v1/auth/forgot-password

Demander une réinitialisation

Envoie un email avec un lien de réinitialisation (valide 1h).

Corps de la requête (JSON)

Champ	Type		Description
email	string	● req	Email du compte

Exemple de réponse

```
{ "message": "Reset email sent if account exists" }
```

■ Réponse identique que le compte existe ou non — sécurité anti-énumération.

POST /api/v1/auth/reset-password

Réinitialiser le mot de passe

Applique le nouveau mot de passe avec le token reçu par email.

Corps de la requête (JSON)

Champ	Type		Description
token	string	● req	Token reçu dans l'email de reset
new_password	string	● req	Nouveau mot de passe

Exemple de réponse

```
{ "message": "Password reset successfully" }
```

POST /api/v1/auth/verify-email

Vérifier l'adresse email

Valide le compte avec le token reçu par email après inscription.

Corps de la requête (JSON)

Champ	Type		Description
token	string	● req	Token de vérification reçu par email

Exemple de réponse

```
{ "message": "Email verified", "access_token": "eyJ..." }
```

■ Frontend : Récupérer le token depuis l'URL (?token=...) et l'envoyer automatiquement.

GET /api/v1/auth/sessions

Lister les sessions actives

Retourne toutes les sessions ouvertes pour l'utilisateur (appareils connectés).

Exemple de réponse

```
{ "sessions": [
  { "id": "sess_01J...", "ip": "41.202.x.x", "device": "Chrome · macOS",
    "created_at": "2026-06-01", "current": true },
  { "id": "sess_02J...", "ip": "196.x.x.x", "device": "Mobile · Android",
```

```
"current": false }  
] }
```

■ Frontend : Afficher dans Paramètres > Sécurité. Permettre de révoquer chaque session.

DELETE /api/v1/auth/sessions/{session_id}

Révoquer une session

Déconnecte un appareil spécifique en invalidant sa session.

Paramètres URL / Query

Champ	Type		Description
session_id	string (path)	● req	ID de la session à révoquer

Exemple de réponse

```
{ "message": "Session revoked" }
```

POST /api/v1/auth/revoke-all-sessions

Révoquer toutes les sessions

Déconnecte tous les appareils sauf la session courante (optionnel).

Corps de la requête (JSON)

Champ	Type		Description
except_current	boolean	■ opt	Si true, garde la session courante active (défaut: false)

Exemple de réponse

```
{ "revoked_count": 4, "message": "All sessions revoked" }
```

DELETE /api/v1/auth/me

Supprimer son compte

Supprime définitivement le compte et toutes les données associées.

Corps de la requête (JSON)

Champ	Type		Description
password	string	● req	Confirmation par le mot de passe

Exemple de réponse

```
{ "message": "Account deleted" }
```

■ Backend : Soft delete (champ deleted_at). RGPD : purge des données personnelles sous 30j.

Organisations & Workspaces

Multi-tenant : orgs, membres, rôles et espaces de travail

POST /api/v1/orgs

Créer une organisation

Crée une nouvelle organisation. Le créateur devient automatiquement owner.

Corps de la requête (JSON)

Champ	Type		Description
name	string	● req	Nom de l'organisation
slug	string	■ opt	Identifiant URL unique (auto-généré si absent)
plan	string	■ opt	"free" "pro" "enterprise" (défaut: free)

Exemple de réponse

```
{ "id": "org_01J...", "name": "Acme Corp", "slug": "acme-corp", "plan": "free" }
```

GET /api/v1/orgs

Lister mes organisations

Retourne toutes les organisations dont l'utilisateur est membre.

Exemple de réponse

```
{ "orgs": [{ "id": "org_01J...", "name": "Acme", "role": "owner", "members_count": 5 } ] }
```

■ Frontend : Utiliser pour le switcher d'organisation dans la navbar.

GET /api/v1/orgs/{org_id}

Détail d'une organisation

Retourne les infos complètes d'une organisation (rôle requis : member+).

Exemple de réponse

```
{ "id": "org_01J...", "name": "Acme Corp", "plan": "pro", "storage_used_mb": 240, "members_count": 8 }
```

PATCH /api/v1/orgs/{org_id}

Modifier l'organisation

Met à jour le nom ou les paramètres. Rôle requis : owner.

Corps de la requête (JSON)

Champ	Type		Description
name	string	■ opt	Nouveau nom
settings	object	■ opt	{ "allow_public_pipelines": bool, "default_timezone": "UTC" }

DELETE /api/v1/orgs/{org_id}

Supprimer l'organisation

Supprime l'organisation et toutes ses données. Irréversible. Rôle : owner.

■ Backend : Vérifier qu'aucun pipeline actif ne tourne avant suppression.

POST**/api/v1/orgs/{org_id}/members/invite****Inviter un membre**

Envoie une invitation par email. L'invité rejoint avec le rôle spécifié.

Corps de la requête (JSON)

Champ	Type		Description
email	string	● req	Email de la personne à inviter
role	string	● req	"viewer" "editor" "admin" "owner"

Exemple de réponse

```
{ "invite_id": "inv_01J...", "email": "bob@acme.com", "expires_at": "2026-06-08" }
```

GET**/api/v1/orgs/{org_id}/members****Lister les membres**

Retourne tous les membres avec leur rôle.

Exemple de réponse

```
{ "members": [{ "user_id": "usr_01J...", "name": "John", "role": "owner", "joined_at": "2026-01-01" }] }
```

PATCH**/api/v1/orgs/{org_id}/members/{user_id}****Changer le rôle d'un membre**

Modifie le rôle d'un membre existant. Rôle requis : admin+.

Corps de la requête (JSON)

Champ	Type		Description
role	string	● req	"viewer" "editor" "admin"

DELETE**/api/v1/orgs/{org_id}/members/{user_id}****Retirer un membre**

Supprime un membre de l'organisation. Le membre perd l'accès immédiatement.

POST**/api/v1/orgs/{org_id}/members/accept-invite****Accepter une invitation**

L'invité accepte son invitation et rejoint l'organisation.

Corps de la requête (JSON)

Champ	Type		Description
token	string	● req	Token d'invitation reçu par email

GET**/api/v1/orgs/{org_id}/workspaces****Lister les workspaces**

Retourne les espaces de travail de l'organisation accessibles à l'utilisateur.

Exemple de réponse

```
{ "workspaces": [{ "id": "ws_01J...", "name": "Production", "pipelines_count": 12 }] }
```


POST**/api/v1/orgs/{org_id}/workspaces****Créer un workspace**

Crée un espace de travail pour isoler les pipelines par équipe ou environnement.

Corps de la requête (JSON)

Champ	Type		Description
name	string	● req	Nom du workspace (ex: Production, Staging)
description	string	■ opt	Description
color	string	■ opt	Couleur hex pour identifier visuellement le workspace

PATCH**/api/v1/orgs/{org_id}/workspaces/{ws_id}****Modifier un workspace**

Met à jour le nom, la description ou la couleur.

DELETE**/api/v1/orgs/{org_id}/workspaces/{ws_id}****Supprimer un workspace**

Supprime le workspace et tous ses pipelines. Irréversible.

■ Backend : Vérifier aucun scheduled run actif. Soft-delete avec période de grâce de 7j.

Pipelines — CRUD & Versioning

Création, gestion, duplication, templates et import/export

GET /api/v1/pipelines

Lister les pipelines

Retourne la liste paginée des pipelines du workspace courant.

Paramètres URL / Query

Champ	Type		Description
workspace_id	string	● req	ID du workspace
page	int	■ opt	Page (défaut: 1)
per_page	int	■ opt	Par page max 50 (défaut: 20)
search	string	■ opt	Recherche sur le nom
status	string	■ opt	"active" "archived"
sort	string	■ opt	"name" "updated_at" "last_run_at"

Exemple de réponse

```
{ "data": [{ "id": "pip_01J...", "name": "Pipeline Transactions",
"status": "active", "nodes_count": 6,
"last_run_at": "2026-06-01T14:32:00Z",
"last_run_status": "success" }],
"pagination": { "total": 42, "page": 1, "per_page": 20 } }
```

■ Frontend : Afficher dans le dashboard. Utiliser le champ last_run_status pour le badge de statut.

POST /api/v1/pipelines

Créer un pipeline

Crée un nouveau pipeline vide dans le workspace.

Corps de la requête (JSON)

Champ	Type		Description
name	string	● req	Nom du pipeline
workspace_id	string	● req	ID du workspace
description	string	■ opt	Description
tags	array	■ opt	Tags pour organiser ["banque", "mensuel"]

Exemple de réponse

```
{ "id": "pip_01J...", "name": "Mon Pipeline", "nodes": [], "edges": [] }
```

■ Frontend : Après création, rediriger vers /pipelines/{id}/editor.

GET /api/v1/pipelines/{pipeline_id}

Charger un pipeline complet

Retourne le pipeline avec tous ses nœuds, edges et leurs configurations.

Exemple de réponse

```
{ "id": "pip_01J...", "name": "Pipeline Transactions",
  "nodes": [{ "id": "node_1", "type": "csv_import",
    "position": { "x": 100, "y": 200 },
    "data": { "config": { "separator": ",", " " } } } ],
  "edges": [{ "id": "edge_1", "source": "node_1", "target": "node_2" } ],
  "version": 14, "updated_at": "2026-06-01T10:00:00Z" }
```

■ **Frontend** : Appeler à l'ouverture de l'éditeur. Injecter nodes et edges dans React Flow directement.

■ **Backend** : Retourner le schéma React Flow natif pour zéro transformation côté frontend.

PUT /api/v1/pipelines/{pipeline_id}

Remplacer un pipeline (sauvegarde complète)

Remplace l'intégralité du pipeline (nodes + edges + config). Crée une nouvelle version.

Corps de la requête (JSON)

Champ	Type		Description
name	string	■ opt	Nouveau nom
nodes	array	● req	Array complet des nœuds (format React Flow)
edges	array	● req	Array complet des edges (format React Flow)

Exemple de réponse

```
{ "id": "pip_01J...", "version": 15, "updated_at": "2026-06-01T10:05:00Z" }
```

■ **Backend** : Snapshot auto de la version précédente avant remplacement. Stocker dans pipeline_versions.

PATCH /api/v1/pipelines/{pipeline_id}

Mise à jour partielle

Modifie seulement les champs fournis (nom, description, tags).

Corps de la requête (JSON)

Champ	Type		Description
name	string	■ opt	Nouveau nom
description	string	■ opt	Description
tags	array	■ opt	Tags

DELETE /api/v1/pipelines/{pipeline_id}

Supprimer un pipeline

Soft-delete. Le pipeline passe en statut "deleted" et disparaît du listing.

■ **Backend** : Soft delete uniquement. Hard delete après 30j si non restauré.

POST /api/v1/pipelines/{pipeline_id}/duplicate

Dupliquer un pipeline

Crée une copie exacte du pipeline avec un nouveau nom.

Corps de la requête (JSON)

Champ	Type		Description
name	string	■ opt	Nom du duplicata (défaut: "Copie de [nom]")

Exemple de réponse

```
{ "id": "pip_02J...", "name": "Copie de Pipeline Transactions" }
```

POST /api/v1/pipelines/{pipeline_id}/archive Archiver un pipeline

Passer le pipeline en statut archivé. Les scheduled runs sont suspendus.

Exemple de réponse

```
{ "status": "archived", "schedules_suspended": 2 }
```

POST /api/v1/pipelines/{pipeline_id}/restore Restaurer un pipeline

Restaurer un pipeline archivé ou supprimé.

GET /api/v1/pipelines/{pipeline_id}/versions Historique des versions

Liste toutes les versions sauvegardées du pipeline.

Exemple de réponse

```
{ "versions": [
  { "id": "ver_01J...", "version": 15, "author": "John",
    "created_at": "2026-06-01T10:05:00Z", "nodes_count": 6 },
  { "id": "ver_02J...", "version": 14, "author": "John",
    "created_at": "2026-06-01T09:30:00Z", "nodes_count": 5 }
] }
```

GET /api/v1/pipelines/{pipeline_id}/versions/{version_id} Charger une version spécifique

Retourne le snapshot complet d'une version passée (nodes + edges).

■ Frontend : Permettre la comparaison côté UI entre version courante et ancienne.

POST /api/v1/pipelines/{pipeline_id}/versions/{version_id}/restore Restaurer une version

Ramène le pipeline à un état antérieur. La version courante est sauvegardée avant.

POST /api/v1/pipelines/{pipeline_id}/versions/snapshot Créer un snapshot manuel

Sauvegarde explicite de la version courante avec un label personnalisé.

Corps de la requête (JSON)

Champ	Type		Description
label	string	■ opt	Label descriptif ex: "Avant refacto du join"

GET /api/v1/pipelines/templates Bibliothèque de templates

Retourne les pipelines pré-construits disponibles.

Exemple de réponse

```
{ "templates": [{ "id": "tpl_banque", "name": "Réconciliation Bancaire",
  "description": "CSV → Filtre → Join → Agrégation → Export",
  "nodes_count": 5, "category": "finance", "preview_url": "..."}] }
```

■ Frontend : Afficher dans un modal "Nouveau pipeline" avec previews visuels.

POST `/api/v1/pipelines/templates/{template_id}/instantiate` Créer un pipeline depuis un template

Génère un pipeline complet basé sur un template. Les nœuds sont pré-configurés.

Corps de la requête (JSON)

Champ	Type		Description
name	string	● req	Nom du pipeline créé
workspace_id	string	● req	Workspace cible

POST `/api/v1/pipelines/import` Importer un pipeline

Importe un pipeline depuis un fichier JSON ou YAML exporté.

Corps de la requête (JSON)

Champ	Type		Description
pipeline_json	object	● req	Pipeline complet exporté (format DataPipe)

GET `/api/v1/pipelines/{pipeline_id}/export` Exporter un pipeline

Retourne le pipeline en JSON portable (sans les données, juste la structure).

Paramètres URL / Query

Champ	Type		Description
format	string	■ opt	"json" "yaml" (défaut: json)

POST `/api/v1/pipelines/{pipeline_id}/publish` Rendre un pipeline public

Publie le pipeline en lecture seule dans la galerie publique.

Nœuds & Edges

Gestion fine des nœuds, connexions et catalogue de types

GET

/api/v1/pipelines/{pipeline_id}/nodes

Lister les nœuds d'un pipeline

Retourne tous les nœuds avec leur position et configuration.

POST

/api/v1/pipelines/{pipeline_id}/nodes

Ajouter un nœud

Ajoute un nouveau nœud au pipeline. Config initiale vide par défaut.

Corps de la requête (JSON)

Champ	Type		Description
type	string	● req	Slug du type : "csv_import" "filter" "join" "aggregate" "ai_transform" ...
position	object	● req	{ "x": 200, "y": 150 } — position sur le canvas
data	object	■ opt	{ "config": {} } — config initiale du nœud
label	string	■ opt	Label personnalisé affiché sur le nœud

Exemple de réponse

```
{ "id": "node_03J...", "type": "filter",  
  "position": { "x": 200, "y": 150 },  
  "data": { "label": "Filtre", "config": {}, "status": "idle" } }
```

■ Frontend : Appeler quand l'utilisateur drop un nœud depuis le panel. Utiliser l'id retourné comme nodeId React Flow.

■ Backend : Valider que type_slug existe dans le catalogue node_types.

GET

/api/v1/pipelines/{pipeline_id}/nodes/{node_id}

Charger un nœud

Retourne la config complète d'un nœud spécifique.

■ Frontend : Appeler quand l'utilisateur clique sur un nœud pour ouvrir l'Inspector.

PUT

/api/v1/pipelines/{pipeline_id}/nodes/{node_id}

Remplacer la config d'un nœud

Remplace entièrement la configuration d'un nœud.

Corps de la requête (JSON)

Champ	Type		Description
position	object	■ opt	Nouvelle position { "x", "y" }
data	object	● req	Nouvelle config complète du nœud

PATCH

`/api/v1/pipelines/{pipeline_id}/nodes/{node_id}`

Mise à jour partielle d'un nœud

Modifie un ou plusieurs champs sans écraser le reste.

Corps de la requête (JSON)

Champ	Type		Description
<code>position</code>	object	■ opt	Déplacer le nœud sur le canvas
<code>data.config</code>	object	■ opt	Mettre à jour la config (merge)
<code>data.label</code>	string	■ opt	Changer le label

■ Frontend : Appeler à chaque `onNodeDragStop` (`position`) et `onConfigChange` (`config`). Debounce 300ms recommandé.

DELETE

`/api/v1/pipelines/{pipeline_id}/nodes/{node_id}`

Supprimer un nœud

Supprime le nœud et tous les edges qui lui sont connectés.

Exemple de réponse

```
{ "deleted_node_id": "node_03J...", "deleted_edges": ["edge_01", "edge_02"] }
```

■ Frontend : React Flow gère la suppression visuelle. Appeler ensuite cet endpoint pour persister.

POST

`/api/v1/pipelines/{pipeline_id}/nodes/bulk`

Créer plusieurs nœuds d'un coup

Permet de coller plusieurs nœuds (copier/coller ou instantiation depuis template).

Corps de la requête (JSON)

Champ	Type		Description
<code>nodes</code>	array	● req	Array de nœuds à créer (même format que POST /nodes)

Exemple de réponse

```
{ "created": [{ "id": "node_04J...", ... }, { "id": "node_05J...", ... }] }
```

GET

`/api/v1/pipelines/{pipeline_id}/edges`

Lister les connexions

Retourne tous les edges (connexions entre nœuds).

Exemple de réponse

```
{ "edges": [{ "id": "edge_01", "source": "node_1", "target": "node_2" }] }
```

POST

`/api/v1/pipelines/{pipeline_id}/edges`

Créer une connexion

Connecte deux nœuds. Le backend valide la compatibilité de types.

Corps de la requête (JSON)

Champ	Type		Description
<code>source</code>	string	● req	ID du nœud source
<code>target</code>	string	● req	ID du nœud cible

source_handle	string	■ opt	Handle de sortie du nœud source
target_handle	string	■ opt	Handle d'entrée du nœud cible

Exemple de réponse

```
{ "id": "edge_03J...", "source": "node_1", "target": "node_3", "valid": true }
```

■ Backend : Vérifier pas de cycle. Vérifier que source_type est compatible avec target_input_type.

DELETE /api/v1/pipelines/{pipeline_id}/edges/{edge_id} Supprimer une connexion

Déconnecte deux nœuds.

Exemple de réponse

```
{ "deleted_edge_id": "edge_03J..." }
```

POST /api/v1/pipelines/{pipeline_id}/edges/validate Valider une connexion

Vérifie qu'un edge est légal sans le créer (pas de cycle, types compatibles).

Corps de la requête (JSON)

Champ	Type		Description
source	string	● req	ID du nœud source
target	string	● req	ID du nœud cible

Exemple de réponse

```
{ "valid": false, "reason": "Cycle detected: node_1 → node_3 → node_1" }
```

■ Frontend : Appeler avant de créer un edge pour afficher un message d'erreur immédiat.

GET /api/v1/node-types Catalogue de tous les types de nœuds

Retourne les types disponibles avec icône, catégorie et description.

Exemple de réponse

```
{ "node_types": [
  { "slug": "csv_import", "label": "CSV Import", "category": "source",
    "color": "#00e5a0", "icon": "file", "description": "Importe un fichier .csv",
    "inputs": 0, "outputs": 1 },
  { "slug": "filter", "label": "Filtre", "category": "transform", ... }
] }
```

■ Frontend : Appeler au montage de l'app. Utiliser pour construire le NodePanel de gauche.

■ Backend : Données statiques ou semi-statiques. Mettre en cache Redis 1h.

GET /api/v1/node-types/{type_slug} Détail d'un type de nœud

Retourne la description complète d'un type (champs, validations, handles).

GET /api/v1/node-types/{type_slug}/schema Schéma de configuration d'un type

Exécution & Runs

Lancer un pipeline, suivre l'exécution en temps réel, annuler

POST `/api/v1/pipelines/{pipeline_id}/execute` Exécuter un pipeline

Lance l'exécution complète du pipeline. Retourne un `run_id` immédiatement (asynchrone).

Corps de la requête (JSON)

Champ	Type		Description
<code>mode</code>	string	■ opt	"full" "from_node" (défaut: full)
<code>from_node</code>	string	■ opt	ID du nœud de départ si mode=from_node
<code>params</code>	object	■ opt	Paramètres dynamiques injectés dans le run

Exemple de réponse

```
{ "run_id": "run_01J...", "status": "queued",  
  "pipeline_id": "pip_01J...", "started_at": "2026-06-01T14:00:00Z" }
```

■ **Frontend** : Dès la réception du `run_id`, ouvrir la WebSocket `/ws/runs/{run_id}` pour le suivi temps réel.

■ **Backend** : Placer en file Celery/ARQ. Retourner `run_id` sans attendre la fin.

POST `/api/v1/pipelines/{pipeline_id}/execute/dry-run` Exécution de validation (dry run)

Valide le pipeline sans exécuter les transformations lourdes. Vérifie la config et les connexions.

Exemple de réponse

```
{ "valid": true, "warnings": [],  
  "estimated_duration_s": 12,  
  "estimated_rows_processed": 1247 }
```

■ **Frontend** : Appeler avant chaque run réel. Afficher les warnings dans l'UI.

POST `/api/v1/pipelines/{pipeline_id}/execute/node/{node_id}` Exécuter un seul nœud

Exécute un nœud spécifique avec les données du nœud précédent. Utile en debug.

Corps de la requête (JSON)

Champ	Type		Description
<code>use_pinned_data</code>	boolean	■ opt	Utiliser les données épinglées si disponibles (défaut: true)

GET `/api/v1/runs` Historique des runs

Retourne l'historique paginé des exécutions.

Paramètres URL / Query

Champ	Type		Description
-------	------	--	-------------

pipeline_id	string	■ opt	Filtrer par pipeline
status	string	■ opt	"queued" "running" "success" "failed" "cancelled"
page	int	■ opt	Page (défaut: 1)
per_page	int	■ opt	Par page (défaut: 20)

Exemple de réponse

```
{ "data": [{ "id": "run_01J...", "status": "success",
"duration_ms": 3420, "rows_processed": 298,
"started_at": "2026-06-01T14:00:00Z" }],
"pagination": { "total": 87 } }
```

GET

/api/v1/runs/{run_id}

Détail d'un run

Retourne le statut global et les métriques d'un run spécifique.

Exemple de réponse

```
{ "id": "run_01J...", "status": "success",
"pipeline_id": "pip_01J...", "duration_ms": 3420,
"rows_processed": 298, "nodes_executed": 5,
"started_at": "2026-06-01T14:00:00Z",
"finished_at": "2026-06-01T14:00:03Z" }
```

POST

/api/v1/runs/{run_id}/cancel

Annuler un run en cours

Stoppe l'exécution d'un run en cours. Les nœuds déjà terminés gardent leurs résultats.

Exemple de réponse

```
{ "status": "cancelled", "cancelled_at": "2026-06-01T14:00:02Z" }
```

■ Frontend : Afficher un bouton Annuler dans la navbar quand un run est actif.

POST

/api/v1/runs/{run_id}/retry

Relancer un run échoué

Rejoue le run depuis le nœud qui a échoué (pas depuis le début).

Corps de la requête (JSON)

Champ	Type		Description
from_beginning	boolean	■ opt	Si true, relance depuis le début (défaut: false)

Exemple de réponse

```
{ "new_run_id": "run_02J...", "status": "queued" }
```

DELETE

/api/v1/runs/{run_id}

Supprimer un run

Supprime le run et ses résultats associés (libère du stockage).

GET

/api/v1/runs/{run_id}/nodes

État de chaque nœud dans un run

Retourne le statut d'exécution de chaque nœud pour ce run.

Exemple de réponse

```
{ "nodes": [
  { "node_id": "node_1", "status": "success", "duration_ms": 120, "rows_out": 1247 },
  { "node_id": "node_2", "status": "success", "duration_ms": 89, "rows_out": 342 },
  { "node_id": "node_3", "status": "running", "started_at": "..." }
] }
```

GET `/api/v1/runs/{run_id}/nodes/{node_id}`

État d'un nœud spécifique

Statut détaillé d'un nœud dans un run.

GET `/api/v1/runs/{run_id}/nodes/{node_id}/pre
view`

Aperçu des données (50 premières lignes)

Retourne un échantillon du DataFrame produit par ce nœud.

Paramètres URL / Query

Champ	Type		Description
limit	int	■ opt	Nombre de lignes retournées (défaut: 50, max: 500)
offset	int	■ opt	Pagination des lignes

Exemple de réponse

```
{ "columns": ["region", "total_montant", "nb_transactions"],
  "rows": [[{"Dakar", 4250000, 87}, {"Abidjan", 3100000, 64}],
  "total_rows": 298, "truncated": true }
```

■ **Frontend** : Utiliser pour remplir la DataTable dans la Console en bas de l'interface.

GET `/api/v1/runs/{run_id}/nodes/{node_id}/sta
ts`

Statistiques d'un nœud

Métriques d'exécution : durée, volume, types de colonnes.

Exemple de réponse

```
{ "rows_in": 342, "rows_out": 298, "columns_count": 3,
  "size_bytes": 14200, "duration_ms": 45,
  "schema": [{ "name": "region", "type": "VARCHAR" }] }
```

■ **Frontend** : Afficher dans le badge sous chaque nœud après exécution. Ex: "298 lignes · 45ms"

GET `/api/v1/runs/{run_id}/logs`

Logs bruts du run

Retourne tous les logs du run (INFO, WARNING, ERROR).

Exemple de réponse

```
{ "logs": [
  { "level": "INFO", "ts": "14:00:00.120", "node_id": "node_1", "msg": "1247 rows loaded" },
  { "level": "WARNING", "ts": "14:00:00.890", "node_id": "node_2", "msg": "12 null values ignored" }
] }
```

SSE `/api/v1/runs/{run_id}/logs/stream`

Logs en streaming (Server-Sent Events)

Stream temps réel des logs pendant l'exécution.

Exemple de réponse

```
data: {"level":"INFO","ts":"14:00:00","node_id":"node_1","msg":"Loading CSV..."}
data: {"level":"INFO","ts":"14:00:00","node_id":"node_1","msg":"1247 rows loaded"}
data: {"level":"INFO","ts":"14:00:01","node_id":"node_2","msg":"Filter applied: 342 rows"}
```

■ **Frontend** : Utiliser `EventSource API`. Afficher dans l'onglet `Logs` de la `Console`.

■ **Backend** : Utiliser `FastAPI StreamingResponse` avec `media_type="text/event-stream"`.

WS /ws/runs/{run_id}

WebSocket — Suivi temps réel du run

Connexion WebSocket qui envoie un event à chaque changement de statut de nœud.

Exemple de réponse

```
// Message reçu à chaque transition de nœud :
{ "type": "node_status",
  "node_id": "node_2",
  "status": "success",
  "rows_out": 342,
  "duration_ms": 89 }
// Message de fin de run :
{ "type": "run_complete", "status": "success", "total_duration_ms": 3420 }
```

■ **Frontend** : Ouvrir dès le déclenchement du run. Utiliser pour animer les nœuds (`idle`→`running`→`success/error`).

■ **Backend** : Utiliser `FastAPI WebSocket`. Broadcaster via `Redis Pub/Sub` si `multi-workers`.

Fichiers & Datasources

Upload de fichiers, connexions DB, inspection de schéma

POST /api/v1/files/upload

Uploader un fichier

Upload d'un fichier CSV, JSON ou XLSX. Multipart form-data requis.

Corps de la requête (JSON)

Champ	Type		Description
file	file	● req	Fichier CSV/JSON/XLSX (max 100MB)
encoding	string	■ opt	Encodage (défaut: auto-detect)
separator	string	■ opt	Séparateur CSV (défaut: auto-detect)

Exemple de réponse

```
{ "id": "file_01J...", "name": "transactions.csv",
  "size_bytes": 245000, "rows": 1247, "columns": 8,
  "schema": [
    { "name": "id", "type": "INTEGER" },
    { "name": "montant", "type": "FLOAT" },
    { "name": "region", "type": "VARCHAR" }
  ],
  "preview_url": "/api/v1/files/file_01J.../preview" }
```

■ Upload multipart/form-data, pas JSON.

■ Auto-détection de l'encodage et du séparateur.

■ Frontend : Afficher une barre de progression pendant l'upload. Afficher le schéma détecté dans l'Inspector.

■ Backend : Parser avec DuckDB directement. Stocker en local filesystem ou S3.

GET /api/v1/files

Lister les fichiers uploadés

Retourne les fichiers disponibles dans le workspace.

Exemple de réponse

```
{ "files": [{ "id": "file_01J...", "name": "transactions.csv", "size_bytes": 245000, "created_at": "..." }] }
```

GET /api/v1/files/{file_id}

Détail d'un fichier

Métadonnées complètes du fichier.

DELETE /api/v1/files/{file_id}

Supprimer un fichier

Supprime le fichier et libère l'espace de stockage.

■ Backend : Vérifier qu'aucun pipeline actif n'utilise ce fichier avant suppression.

GET

/api/v1/files/{file_id}/schema

Schéma détecté du fichier

Retourne les colonnes et types inférés automatiquement.

Exemple de réponse

```
{ "columns": [
  { "name": "id", "type": "INTEGER", "nullable": false, "sample": "1" },
  { "name": "montant", "type": "FLOAT", "nullable": false, "sample": "75000.0" },
  { "name": "region", "type": "VARCHAR", "nullable": true, "sample": "Dakar" },
  { "name": "date_tx", "type": "DATE", "nullable": false, "sample": "2026-01-15" }
] }
```

■ Frontend : Utiliser pour peupler les dropdowns de colonnes dans l'Inspector (Filtre, Join, Agrégation).

GET

/api/v1/files/{file_id}/preview

Aperçu des premières lignes

Retourne les 100 premières lignes du fichier.

Paramètres URL / Query

Champ	Type		Description
limit	int	■ opt	Nombre de lignes (défaut: 100, max: 1000)
offset	int	■ opt	Décalage

POST

/api/v1/files/{file_id}/reparse

Re-analyser un fichier

Force une nouvelle détection du schéma avec des paramètres différents.

Corps de la requête (JSON)

Champ	Type		Description
separator	string	■ opt	Nouveau séparateur à utiliser
encoding	string	■ opt	Nouvel encodage
header	boolean	■ opt	La première ligne est-elle un header ?

POST

/api/v1/datasources

Créer une connexion de données

Enregistre une connexion vers une base de données externe.

Corps de la requête (JSON)

Champ	Type		Description
name	string	● req	Nom affiché dans l'UI
type	string	● req	"postgresql" "mysql" "sqlite" "bigquery" "snowflake"
host	string	■ opt	Hôte de la DB (pour SQL)
port	int	■ opt	Port
database	string	■ opt	Nom de la base
username	string	■ opt	Utilisateur

password

string

■ opt

Mot de passe (chiffré en stockage)

Exemple de réponse

```
{ "id": "ds_01J...", "name": "DB Production", "type": "postgresql", "status": "untested" }
```

■ Backend : Ne jamais retourner le mot de passe en clair. AES-256 en DB.

GET

/api/v1/datasources

Lister les datasources

Retourne toutes les connexions du workspace.

PATCH

/api/v1/datasources/{ds_id}

Modifier une datasource

Met à jour la connexion sans exposer le mot de passe actuel.

DELETE

/api/v1/datasources/{ds_id}

Supprimer une datasource

Supprime la connexion. Les pipelines qui l'utilisent sont mis en erreur.

POST

/api/v1/datasources/{ds_id}/test

Tester une connexion

Vérifie que la connexion DB est accessible (SELECT 1).

Exemple de réponse

```
{ "success": true, "latency_ms": 34, "server_version": "PostgreSQL 15.2" }
```

■ Frontend : Afficher un bouton "Tester la connexion" dans le formulaire de config de datasource.

GET

/api/v1/datasources/{ds_id}/tables

Lister les tables d'une datasource

Retourne les tables/vues disponibles dans la base de données.

Exemple de réponse

```
{ "tables": [{ "name": "transactions", "schema": "public", "rows_estimate": 50000 }] }
```

GET

/api/v1/datasources/{ds_id}/tables/{table}/schema

Schéma d'une table

Retourne les colonnes et types d'une table de la datasource.

GET

/api/v1/datasources/{ds_id}/tables/{table}/preview

Aperçu d'une table

Retourne les 100 premières lignes de la table.

POST

/api/v1/datasources/{ds_id}/query

Requête libre sur la datasource

Exécute une requête SQL en lecture seule sur la datasource.

Corps de la requête (JSON)

Champ	Type		Description
sql	string	● req	Requête SQL (SELECT uniquement)
limit	int	■ opt	Limite de lignes retournées (défaut: 500)

■ Backend : Valider que la requête est en lecture seule (pas de INSERT/UPDATE/DELETE/DROP).

POST

/api/v1/datasources/{ds_id}/sync

Synchroniser les métadonnées

Rafraîchit la liste des tables et leurs schémas.

Transformations SQL

Validation, formatage, preview des opérations DuckDB

POST /api/v1/transform/filter/validate

Valider une expression de filtre

Vérifie qu'une expression de filtre est syntaxiquement correcte sur le schéma donné.

Corps de la requête (JSON)

Champ	Type		Description
expression	string	● req	Expression ex: "montant > 50000"
schema	array	● req	Schéma des colonnes disponibles

Exemple de réponse

```
{ "valid": true, "estimated_matches": 342, "total_rows": 1247 }
// ou si invalide :
{ "valid": false, "error": "Column 'montant2' not found in schema" }
```

■ Frontend : Appeler à chaque modification dans l'input de filtre. Afficher l'estimation en temps réel.

POST /api/v1/transform/sql/validate

Valider une requête SQL

Parse et valide une requête DuckDB sans l'exécuter.

Corps de la requête (JSON)

Champ	Type		Description
sql	string	● req	Requête SQL DuckDB
schema	array	■ opt	Schéma de contexte pour la validation des colonnes

Exemple de réponse

```
{ "valid": true, "query_type": "SELECT",
  "referenced_columns": ["montant", "region"],
  "warnings": ["Column 'date' is ambiguous"] }
```

POST /api/v1/transform/sql/explain

EXPLAIN ANALYZE DuckDB

Retourne le plan d'exécution estimé d'une requête DuckDB.

Corps de la requête (JSON)

Champ	Type		Description
sql	string	● req	Requête SQL
data	array	■ opt	Échantillon de données pour l'analyse

Exemple de réponse

```
{ "plan": "HASH_JOIN → FILTER → SEQ_SCAN",
  "estimated_rows": 298,
  "estimated_duration_ms": 45 }
```

POST /api/v1/transform/sql/format

Formater le SQL

Reformate une requête SQL selon les conventions DuckDB (indentation, majuscules).

Corps de la requête (JSON)			
Champ	Type		Description
sql	string	● req	Requête SQL à formater

Exemple de réponse

```
{ "formatted": "SELECT\n region,\n SUM(montant) AS total\nFROM transactions\nWHERE montant > 50000\nGROUP BY\n region" }
```

■ Frontend : Bouton "Formater" dans l'éditeur SQL du nœud SQL Query.

POST /api/v1/transform/schema/infer

Inférer le schéma d'un DataFrame

Analyse un échantillon de données et retourne les types inférés.

Corps de la requête (JSON)			
Champ	Type		Description
sample	array	● req	Échantillon de lignes (max 100)

POST /api/v1/transform/schema/cast

Convertir les types de colonnes

Applique un cast de types sur le schéma (ex: string → float).

Corps de la requête (JSON)			
Champ	Type		Description
casts	array	● req	[{"column": "montant", "from": "VARCHAR", "to": "FLOAT"}]

Exemple de réponse

```
{ "sql": "CAST(montant AS FLOAT)" }
```

POST /api/v1/transform/join/preview

Preview d'un join sans exécuter

Retourne un aperçu du résultat d'un join sur des données d'exemple.

Corps de la requête (JSON)			
Champ	Type		Description
left_file_id	string	● req	ID du fichier gauche
right_file_id	string	● req	ID du fichier droit
left_key	string	● req	Colonne clé gauche
right_key	string	● req	Colonne clé droite
join_type	string	● req	"LEFT" "INNER" "OUTER"

Exemple de réponse

```
{ "match_count": 298, "total_left": 342, "total_right": 300,\n "unmatched_left": 44, "sample_rows": [...] }
```

■ Frontend : Afficher dans l'Inspector du nœud Join pour guider le choix des clés.

POST**/api/v1/transform/aggregate/preview****Preview d'une agrégation**

Retourne un aperçu du résultat d'une agrégation.

Corps de la requête (JSON)

Champ	Type		Description
file_id	string	● req	ID du fichier source
group_by	array	● req	Colonnes de regroupement
aggregates	array	● req	[[{"column": "montant", "func": "SUM", "alias": "total"}]]

GET**/api/v1/transform/functions****Fonctions DuckDB supportées**

Retourne la liste des fonctions disponibles pour l'autocomplétion.

Exemple de réponse

```
{ "functions": [  
  { "name": "SUM", "category": "aggregate", "signature": "SUM(expr)", "desc": "Somme" },  
  { "name": "STRFTIME", "category": "date", "signature": "STRFTIME(fmt, date)", "desc": "Formater une date" }  
] }
```

■ **Frontend** : Utiliser pour l'autocomplétion dans l'éditeur SQL (Monaco Editor recommandé).

Intelligence Artificielle

Génération de pipelines, SQL, suggestions et chat contextuel

POST /api/v1/ai/generate/pipeline

Générer un pipeline complet depuis un prompt

Analyse le texte libre et génère un pipeline JSON complet avec nœuds et connexions. C'est le endpoint du bonus IA.

Corps de la requête (JSON)

Champ	Type		Description
prompt	string	● req	Description en langage naturel (français ou anglais)
schema	array	■ opt	Schéma des fichiers disponibles pour enrichir la génération
context	object	■ opt	{ "existing_nodes": [...] } pour compléter un pipeline partiel

Exemple de réponse

```
{ "pipeline": {
  "nodes": [
    { "type": "csv_import", "position": { "x": 100, "y": 200 },
    "data": { "config": { "separator": "," } } },
    { "type": "filter", "position": { "x": 320, "y": 200 },
    "data": { "config": { "column": "montant", "op": ">", "value": "50000" } } },
    { "type": "aggregate", "position": { "x": 540, "y": 200 },
    "data": { "config": { "group_by": ["region"], "funcs": [{"col": "montant", "fn": "SUM"}] } } }
  ],
  "edges": [
    { "source": "node_0", "target": "node_1" },
    { "source": "node_1", "target": "node_2" }
  ]
},
"explanation": "Pipeline de 3 étapes : import CSV → filtre montant > 50k → agrégation par région",
"tokens_used": 842 }
```

- Ce endpoint consomme des tokens Claude. Inclus dans le quota mensuel de l'org.
- Frontend : Injecter la réponse directement dans React Flow (setNodes + setEdges). Animer l'apparition des nœuds.
- Backend : System prompt : décrire précisément le format JSON React Flow attendu. Température 0 pour la cohérence.

POST /api/v1/ai/generate/sql

Générer une requête SQL depuis un prompt

Génère du SQL DuckDB à partir d'une description textuelle et d'un schéma.

Corps de la requête (JSON)

Champ	Type		Description
prompt	string	● req	Description de la requête souhaitée
schema	array	● req	Schéma des tables disponibles
dialect	string	■ opt	"duckdb" "postgresql" (défaut: duckdb)

Exemple de réponse

```
{ "sql": "SELECT region, SUM(montant) AS total_montant, COUNT(*) AS nb_tx\nFROM transactions t\nLEFT JOIN clients c ON t.client_id = c.id\nWHERE t.montant > 50000\nGROUP BY region\nORDER BY total_montant DESC",\n"explanation": "Jointure LEFT avec clients, filtre montant, agrégation par région",\n"tokens_used": 312 }
```

■ Frontend : Afficher le SQL généré dans l'éditeur Monaco avec coloration syntaxique. Permettre l'édition avant exécution.

POST /api/v1/ai/generate/filter

Générer une expression de filtre

Génère une expression de filtre DuckDB depuis une description naturelle.

Corps de la requête (JSON)

Champ	Type		Description
prompt	string	● req	Ex: "transactions au-dessus de 50k FCFA ce mois-ci"
columns	array	● req	Colonnes disponibles pour le filtre

Exemple de réponse

```
{ "expression": "montant > 50000 AND date_tx >= '2026-06-01'",\n"columns_used": ["montant", "date_tx"] }
```

POST /api/v1/ai/suggest/columns

Suggérer les colonnes pertinentes

Pour un type de nœud donné, suggère quelles colonnes du schéma utiliser.

Corps de la requête (JSON)

Champ	Type		Description
node_type	string	● req	Type du nœud : "filter" "aggregate" "join"
schema	array	● req	Schéma des colonnes disponibles

Exemple de réponse

```
{ "suggestions": [\n{\n"column": "montant",\n"confidence": 0.95,\n"reason": "Colonne numérique, idéale pour filtrage/agrégation"\n},\n{\n"column": "region",\n"confidence": 0.88,\n"reason": "Colonne catégorielle, idéale pour GROUP BY"\n},\n{\n"column": "client_id",\n"confidence": 0.82,\n"reason": "Colonne ID, idéale pour JOIN"\n}\n]
```

■ Frontend : Afficher comme suggestions dans les dropdowns de l'Inspector. Badge de confiance optionnel.

POST /api/v1/ai/suggest/joins

Suggérer les clés de jointure

Analyse deux schémas et suggère les paires de colonnes probables pour un join.

Corps de la requête (JSON)

Champ	Type		Description
schema_left	array	● req	Schéma du dataset de gauche
schema_right	array	● req	Schéma du dataset de droite

Exemple de réponse

```
{ "suggestions": [\n{\n"left": "client_id",\n"right": "id",\n"confidence": 0.97,\n"reason": "Noms similaires + types compatibles INTEGER"\n},\n{\n"left": "region",\n"right": "region_code",\n"confidence": 0.71,\n"reason": "Contenu similaire VARCHAR"\n}\n]
```

■ Frontend : Pré-remplir les champs de clé de jointure dans l'Inspector du nœud Join.

POST /api/v1/ai/suggest/pipeline

Suggérer un pipeline selon l'objectif

À partir d'un objectif métier et de fichiers disponibles, propose une architecture de pipeline.

Corps de la requête (JSON)

Champ	Type		Description
goal	string	● req	Ex: "Je veux analyser les ventes par région ce trimestre"
available_files	array	● req	IDs et schémas des fichiers disponibles

Exemple de réponse

```
{ "suggestion": "Je recommande 5 nœuds : CSV Import → Filtre date → Join clients → Agrégation région → Bar Chart",  
  "reasoning": "Votre objectif nécessite un croisement avec la table clients pour avoir les régions.",  
  "generate_now": true }
```

POST /api/v1/ai/explain/sql

Expliquer une requête SQL en langage naturel

Traduit une requête SQL complexe en explication compréhensible.

Corps de la requête (JSON)

Champ	Type		Description
sql	string	● req	Requête SQL à expliquer

Exemple de réponse

```
{ "explanation": "Cette requête récupère les transactions supérieures à 50 000 FCFA, les relie aux informations clients via leur identifiant, puis calcule le total et le nombre de transactions par région géographique, triés du plus grand au plus petit total.",  
  "level": "simple" }
```

■ Frontend : Afficher dans une tooltip ou un panneau à côté de l'éditeur SQL.

POST /api/v1/ai/explain/error

Expliquer une erreur en langage naturel

Traduit un message d'erreur technique en action claire et compréhensible.

Corps de la requête (JSON)

Champ	Type		Description
error_message	string	● req	Message d'erreur brut
context	object	■ opt	{ "node_type": "join", "config": {...} }

Exemple de réponse

```
{ "plain_explanation": "La colonne 'client_id' n'existe pas dans votre fichier CSV. Vérifiez le nom exact dans l'onglet Schéma.",  
  "suggested_fix": "Remplacer 'client_id' par 'id_client' dans la configuration du nœud Join.",  
  "severity": "error" }
```

■ Frontend : Afficher automatiquement quand un nœud passe en statut error. Remplace le message d'erreur brut.

POST /api/v1/ai/chat

Chat libre avec contexte du pipeline

Interface de chat IA avec connaissance du pipeline courant. Permet de poser des questions métier.

Corps de la requête (JSON)

Champ	Type		Description
messages	array	● req	Historique : [{ "role": "user" "assistant", "content": "..."}]
pipeline_context	object	■ opt	{ "pipeline_id": "...", "current_node": "...", "schema": [...] }

Exemple de réponse

```
{ "message": { "role": "assistant", "content": "Pour comparer les performances mois par mois, ajoutez un nœud Agrégation avec GROUP BY STRFTIME('%Y-%m', date_tx). Voulez-vous que je génère ce nœud ?" }, "session_id": "chat_01J...", "tokens_used": 156 }
```

■ Frontend : Implémenter comme un panneau de chat flottant. Maintenir l'historique dans le state React.

GET

/api/v1/ai/history

Historique des sessions IA

Liste les sessions de chat et de génération de l'utilisateur.

DELETE

/api/v1/ai/history/{session_id}

Supprimer une session IA

Supprime l'historique d'une session de chat.

GET

/api/v1/ai/usage

Consommation IA du mois

Retourne les tokens consommés et le quota mensuel.

Exemple de réponse

```
{ "month": "2026-06", "tokens_used": 84200, "tokens_limit": 500000, "cost_usd": 0.84, "breakdown": { "generate_pipeline": 42000, "generate_sql": 18000, "chat": 24200 } }
```

■ Frontend : Afficher dans Paramètres > Usage pour informer l'utilisateur de sa consommation.

Résultats & Exports

Téléchargement des données produites et partage de résultats

GET `/api/v1/runs/{run_id}/result`

Résultat global du run

Retourne le résultat du nœud de sortie final du pipeline.

GET `/api/v1/runs/{run_id}/result/nodes/{node_id}`

Résultat d'un nœud spécifique

Retourne les données produites par un nœud précis dans ce run.

POST `/api/v1/exports`

Créer un export

Prépare un fichier d'export à partir du résultat d'un nœud.

Corps de la requête (JSON)

Champ	Type		Description
<code>run_id</code>	string	● req	ID du run source
<code>node_id</code>	string	● req	ID du nœud dont exporter les données
<code>format</code>	string	● req	"csv" "json" "xlsx" "parquet"
<code>options</code>	object	■ opt	{ "separator": ",", "encoding": "UTF-8", "include_header": true }

Exemple de réponse

```
{ "export_id": "exp_01J...", "status": "preparing",  
  "format": "csv", "estimated_size_bytes": 14200 }
```

■ Backend : Génération asynchrone pour les gros fichiers. Notifier via WebSocket quand prêt.

GET `/api/v1/exports/{export_id}`

Statut d'un export

Vérifie si l'export est prêt au téléchargement.

Exemple de réponse

```
{ "id": "exp_01J...", "status": "ready",  
  "size_bytes": 14200, "rows": 298,  
  "download_url": "/api/v1/exports/exp_01J.../download",  
  "expires_at": "2026-06-08T14:00:00Z" }
```

GET `/api/v1/exports/{export_id}/download`

Télécharger un export

Retourne le fichier exporté en téléchargement direct.

■ Content-Disposition: attachment. Supporte les ranges HTTP pour les gros fichiers.

■ Frontend : Déclencher avec `window.open()` ou un lien `a[download]`.

DELETE /api/v1/exports/{export_id}

Supprimer un export

Supprime le fichier d'export pour libérer l'espace de stockage.

GET /api/v1/exports

Historique des exports

Liste les exports créés dans le workspace.

POST /api/v1/exports/{export_id}/share

Partager un export

Génère un lien public temporaire pour partager un export sans authentification.

Corps de la requête (JSON)

Champ	Type		Description
expires_in_hours	int	■ opt	Durée de validité en heures (défaut: 48)
password	string	■ opt	Mot de passe optionnel pour protéger le lien

Exemple de réponse

```
{ "share_url": "https://datapipe.app/shared/tok_abc123",  
  "expires_at": "2026-06-03T14:00:00Z" }
```

■ Frontend : Bouton "Partager le résultat" dans la console. Afficher le lien avec copie en 1 clic.

GET /api/v1/shared/{share_token}

Accéder à un export partagé (public)

Endpoint public — permet de télécharger un export via un lien partagé sans auth.

■ Endpoint public, pas de Bearer token requis.

■ Si protégé par mot de passe, retourne 401 et attend { "password": "..." } en query param.

POST /api/v1/pipelines/{pipeline_id}/publish

Publier le pipeline (galerie publique)

Rend le pipeline consultable dans la bibliothèque publique de templates.

Scheduling — Exécutions planifiées

Cron jobs, triggers automatiques et historique

GET

/api/v1/pipelines/{pipeline_id}/schedules

Lister les planifications

Retourne les schedules actifs d'un pipeline.

POST

/api/v1/pipelines/{pipeline_id}/schedules

Créer une planification

Planifie l'exécution automatique d'un pipeline selon une expression cron.

Corps de la requête (JSON)

Champ	Type		Description
name	string	● req	Nom de la planification
cron	string	● req	Expression cron ex: "0 8 * * 1" (lundi 8h)
timezone	string	■ opt	Timezone (défaut: UTC) ex: "Africa/Abidjan"
enabled	boolean	■ opt	Activer immédiatement (défaut: true)
notify_on_failure	boolean	■ opt	Email si le run échoue (défaut: true)

Exemple de réponse

```
{ "id": "sched_01J...", "cron": "0 8 * * 1",  
  "next_run_at": "2026-06-09T08:00:00Z", "enabled": true }
```

■ Backend : Utiliser APScheduler ou Celery Beat. Stocker next_run_at calculé.

PATCH

/api/v1/pipelines/{pipeline_id}/schedules
/{sched_id}

Modifier une planification

Met à jour l'expression cron ou le timezone.

DELETE

/api/v1/pipelines/{pipeline_id}/schedules
/{sched_id}

Supprimer une planification

Supprime la planification définitivement.

POST

/api/v1/pipelines/{pipeline_id}/schedules
/{sched_id}/pause

Mettre en pause

Suspend la planification sans la supprimer.

Exemple de réponse

```
{ "status": "paused", "next_run_at": null }
```

POST

/api/v1/pipelines/{pipeline_id}/schedules
/{sched_id}/resume

Reprendre une planification

Réactive une planification mise en pause.

Exemple de réponse

```
{ "status": "active", "next_run_at": "2026-06-09T08:00:00Z" }
```

POST

**/api/v1/pipelines/{pipeline_id}/schedules
/{sched_id}/trigger**

Déclencher manuellement

Lance immédiatement le pipeline comme si le cron s'était déclenché.

Exemple de réponse

```
{ "run_id": "run_05J...", "triggered_by": "manual" }
```

GET

**/api/v1/pipelines/{pipeline_id}/schedules
/{sched_id}/history**

Historique d'une planification

Retourne les dernières exécutions déclenchées par ce schedule.

Exemple de réponse

```
{ "history": [
  { "run_id": "run_04J...", "status": "success", "triggered_at": "2026-06-02T08:00:00Z" },
  { "run_id": "run_03J...", "status": "failed", "triggered_at": "2026-05-26T08:00:00Z" }
] }
```

GET

**/api/v1/pipelines/{pipeline_id}/schedules
/{sched_id}**

Détail d'une planification

Retourne la configuration complète d'un schedule.

Webhooks

Intégrations sortantes et déclencheurs entrants

GET /api/v1/webhooks

Lister les webhooks

Retourne tous les webhooks configurés dans le workspace.

POST /api/v1/webhooks

Créer un webhook sortant

Envoie un POST à une URL externe lors d'événements DataPipe.

Corps de la requête (JSON)

Champ	Type		Description
name	string	● req	Nom du webhook
url	string	● req	URL de destination
events	array	● req	["run.success", "run.failed", "pipeline.created"]
secret	string	■ opt	Secret pour signer les payloads (HMAC-SHA256)
headers	object	■ opt	Headers additionnels à inclure

Exemple de réponse

```
{ "id": "wh_01J...", "url": "https://slack.com/...", "events": ["run.failed"],  
  "active": true, "created_at": "2026-06-01" }
```

■ Backend : Signer chaque payload avec HMAC-SHA256 du secret. Header X-DataPipe-Signature.

PATCH /api/v1/webhooks/{webhook_id}

Modifier un webhook

Met à jour l'URL, les événements ou le secret.

DELETE /api/v1/webhooks/{webhook_id}

Supprimer un webhook

Supprime le webhook définitivement.

POST /api/v1/webhooks/{webhook_id}/test

Tester un webhook

Envoie un event de test à l'URL configurée.

Exemple de réponse

```
{ "delivered": true, "response_status": 200, "response_time_ms": 145 }
```

GET /api/v1/webhooks/{webhook_id}/deliveries

Historique des livraisons

Retourne les derniers envois avec leur statut.

Exemple de réponse

```
{ "deliveries": [
  { "id": "del_01J...", "event": "run.success", "status": 200, "sent_at": "...", "duration_ms": 145 },
  { "id": "del_02J...", "event": "run.failed", "status": 500, "sent_at": "...", "error": "Connection refused"
}
] }
```

POST

/api/v1/webhooks/{webhook_id}/deliveries/{delivery_id}/retry

Réessayer une livraison

Renvoie un event qui avait échoué.

POST

/api/v1/webhooks/inbound/{token}

Déclencher un pipeline depuis l'extérieur (webhook entrant)

Endpoint public qui déclenche un pipeline via un appel HTTP externe (Zapier, Make, etc.).

Corps de la requête (JSON)

Champ	Type		Description
payload	object	■ opt	Données envoyées — accessibles dans le pipeline comme paramètres

Exemple de réponse

```
{ "run_id": "run_06J...", "status": "queued" }
```

■ token est un identifiant unique par pipeline — ne pas exposer publiquement.

■ Frontend : Afficher dans Paramètres du pipeline > Webhook entrant avec bouton de copie de l'URL.

GET

/api/v1/webhooks/{webhook_id}

Détail d'un webhook

Retourne la configuration complète (sans le secret).

Notifications & Alertes

Centre de notifications et règles d'alertes sur métriques

GET `/api/v1/notifications`

Lister les notifications

Retourne les notifications de l'utilisateur (runs, invitations, alertes).

Paramètres URL / Query

Champ	Type		Description
<code>unread_only</code>	boolean	■ opt	Si true, retourne seulement les non-lues
<code>page</code>	int	■ opt	Page

Exemple de réponse

```
{ "notifications": [  
  { "id": "notif_01J...", "type": "run_failed", "read": false,  
    "message": "Pipeline 'Transactions' a échoué",  
    "created_at": "2026-06-01T14:05:00Z",  
    "data": { "pipeline_id": "pip_01J...", "run_id": "run_01J..." } }  
],  
  "unread_count": 3 }
```

■ Frontend : Afficher le badge "unread_count" dans la navbar. Polling toutes les 30s ou WebSocket.

PATCH `/api/v1/notifications/{notif_id}/read`

Marquer comme lue

Passe une notification en statut "lue".

POST `/api/v1/notifications/read-all`

Tout marquer comme lu

Marque toutes les notifications non lues comme lues.

DELETE `/api/v1/notifications/{notif_id}`

Supprimer une notification

Supprime une notification de la liste.

GET `/api/v1/alerts`

Lister les règles d'alerte

Retourne les alertes configurées dans le workspace.

POST `/api/v1/alerts`

Créer une règle d'alerte

Crée une alerte déclenchée sur condition métrique.

Corps de la requête (JSON)

Champ	Type		Description
-------	------	--	-------------

name	string	● req	Nom de l'alerte
pipeline_id	string	● req	Pipeline concerné
condition	string	● req	"run_failed" "duration_gt" "rowcount_lt" "rowcount_gt"
threshold	number	■ opt	Valeur seuil (pour duration_gt et rowcount_*)
channels	array	● req	["email"] ["slack"] ["webhook"]
recipients	array	■ opt	Emails des destinataires

Exemple de réponse

```
{ "id": "alert_01J...", "name": "Alerte volume faible",  
  "condition": "rowcount_lt", "threshold": 100,  
  "enabled": true }
```

GET

/api/v1/alerts/{alert_id}

Détail d'une alerte

Retourne la configuration complète d'une règle d'alerte.

PATCH

/api/v1/alerts/{alert_id}

Modifier une alerte

Met à jour la condition ou les seuils d'une alerte.

DELETE

/api/v1/alerts/{alert_id}

Supprimer une alerte

Supprime la règle d'alerte définitivement.

POST

/api/v1/alerts/{alert_id}/mute

Mettre en silence une alerte

Désactive temporairement une alerte sans la supprimer.

Corps de la requête (JSON)

Champ	Type		Description
until	string	■ opt	ISO datetime de fin du silence (ex: "2026-06-07T08:00:00Z")

Exemple de réponse

```
{ "muted_until": "2026-06-07T08:00:00Z" }
```


Analytics & Audit

Métriques de performance des pipelines et logs d'audit

GET `/api/v1/analytics/pipelines/{pipeline_id}` **Métriques d'un pipeline**

Retourne les statistiques d'exécution du pipeline sur la période.

Paramètres URL / Query

Champ	Type		Description
period	string	■ opt	"7d" "30d" "90d" (défaut: 30d)

Exemple de réponse

```
{ "total_runs": 42, "success_rate": 0.95, "avg_duration_ms": 3200,
  "avg_rows_processed": 1100, "total_rows_processed": 46200,
  "last_run_at": "2026-06-01T14:00:00Z" }
```

GET `/api/v1/analytics/pipelines/{pipeline_id}/runs/trend` **Tendance des runs dans le temps**

Retourne les runs par jour sur la période pour afficher un graphique.

Exemple de réponse

```
{ "trend": [
  { "date": "2026-05-26", "runs": 3, "success": 3, "failed": 0 },
  { "date": "2026-05-27", "runs": 5, "success": 4, "failed": 1 }
] }
```

■ Frontend : Utiliser *Recharts LineChart* pour afficher la tendance dans la page *Analytics*.

GET `/api/v1/analytics/pipelines/{pipeline_id}/nodes/bottlenecks` **Identifier les nœuds lents**

Retourne les nœuds classés par durée moyenne d'exécution.

Exemple de réponse

```
{ "bottlenecks": [
  { "node_id": "node_3", "label": "Join Clients", "avg_duration_ms": 2100, "rank": 1 },
  { "node_id": "node_5", "label": "IA Transform", "avg_duration_ms": 890, "rank": 2 }
] }
```

■ Frontend : Mettre en surbrillance les nœuds lents sur le canvas (*heatmap*).

GET `/api/v1/analytics/workspace/usage` **Usage global du workspace**

Volume de données traité, nombre de runs, espace de stockage.

Exemple de réponse

```
{ "storage_used_mb": 480, "storage_limit_mb": 5000,
  "runs_this_month": 87, "rows_processed_this_month": 142000,
  "active_pipelines": 12, "scheduled_runs": 4 }
```

GET

`/api/v1/analytics/workspace/costs`

Estimation des coûts IA

Tokens consommés et coût estimé en USD pour le mois courant.

GET

`/api/v1/audit-logs`

Logs d'audit

Historique de toutes les actions effectuées dans le workspace (qui a fait quoi, quand).

Paramètres URL / Query

Champ	Type		Description
resource	string	■ opt	"pipeline" "datasource" "member" "api_key"
action	string	■ opt	"create" "update" "delete" "execute"
user_id	string	■ opt	Filtrer par utilisateur
from	string	■ opt	Date de début ISO
to	string	■ opt	Date de fin ISO
page	int	■ opt	Page

Exemple de réponse

```
{ "logs": [  
  { "id": "log_01J...", "user": "John Doe", "action": "delete",  
    "resource": "pipeline", "resource_id": "pip_01J...",  
    "ip": "41.202.x.x", "timestamp": "2026-06-01T09:30:00Z" }  
]
```

■ Backend : Table immutable. INSERT only. Aucun UPDATE/DELETE autorisé sur cette table.

API Keys & Intégrations

Gestion des clés d'API et connecteurs externes

GET /api/v1/api-keys

Lister les clés API

Retourne les clés API de l'organisation (sans les valeurs secrètes).

Exemple de réponse

```
{ "api_keys": [
  { "id": "key_01J...", "name": "Prod Key", "prefix": "dp_prod...",
    "last_used_at": "2026-06-01", "created_at": "2026-01-01",
    "scopes": ["pipelines:read", "runs:write"] }
]
```

POST /api/v1/api-keys

Créer une clé API

Génère une nouvelle clé API avec des scopes limités.

Corps de la requête (JSON)

Champ	Type		Description
name	string	● req	Nom descriptif
scopes	array	● req	["pipelines:read", "pipelines:write", "runs:write", "files:write"]

Exemple de réponse

```
{ "id": "key_02J...", "name": "CI/CD Key",
  "key": "dp_live_XXXXXXXXXXXXXXXXXXXX",
  "scopes": ["runs:write"],
  "warning": "Copiez cette clé maintenant – elle ne sera plus affichée." }
```

■ La valeur de la clé n'est retournée qu'une seule fois à la création.

■ Frontend : Afficher dans un modal avec alerte "Copiez maintenant". Bouton copie en presse-papier.

DELETE /api/v1/api-keys/{key_id}

Révoquer une clé API

Supprime la clé. Toutes les requêtes utilisant cette clé échouent immédiatement.

POST /api/v1/api-keys/{key_id}/rotate

Rotation d'une clé API

Génère une nouvelle valeur pour la clé. L'ancienne est immédiatement invalidée.

Exemple de réponse

```
{ "new_key": "dp_live_yyy...", "rotated_at": "2026-06-01" }
```

GET /api/v1/integrations

Catalogue des intégrations disponibles

Liste les connecteurs disponibles (Slack, Notion, Google Sheets, etc.).

Exemple de réponse

```
{ "integrations": [
  { "slug": "slack", "name": "Slack", "status": "available", "connected": false },
  { "slug": "google-sheets", "name": "Google Sheets", "status": "available", "connected": true },
  { "slug": "notion", "name": "Notion", "status": "beta", "connected": false }
] }
```

POST**/api/v1/integrations/{slug}/connect****Connecter une intégration**

Lance le flux OAuth ou stocke les credentials pour une intégration.

Corps de la requête (JSON)

Champ	Type		Description
auth_code	string	■ opt	Code OAuth si flow OAuth2
credentials	object	■ opt	Credentials manuels { "api_key": "..." }

Exemple de réponse

```
{ "connected": true, "account": "workspace@acme.com" }
```

DELETE**/api/v1/integrations/{slug}/disconnect****Déconnecter une intégration**

Révoque les tokens et supprime les credentials stockés.

GET**/api/v1/integrations/{slug}/status****Statut d'une intégration**

Vérifie que la connexion est toujours valide.

Exemple de réponse

```
{ "connected": true, "healthy": true, "account": "workspace@acme.com" }
```

Santé & Ops

Health checks, métriques Prometheus et infos système

GET /health

Health check global

Vérifie que l'API est opérationnelle. Utilisé par le load balancer.

Exemple de réponse

```
{ "status": "ok", "version": "1.0.0", "uptime_s": 86400 }
```

■ Répondre sous 50ms. Pas d'authentification requise.

GET /health/live

Liveness probe (Kubernetes)

Vérifie que le processus est vivant (pas de deadlock). Retourne 200 si OK.

Exemple de réponse

```
{ "alive": true }
```

GET /health/ready

Readiness probe (Kubernetes)

Vérifie que le service est prêt à recevoir du trafic (DB connectée, Redis OK).

Exemple de réponse

```
{ "ready": true, "checks": { "database": "ok", "redis": "ok", "storage": "ok" } }
```

GET /metrics

Métriques Prometheus

Expose les métriques au format Prometheus pour le monitoring.

Exemple de réponse

```
# HELP http_requests_total Total HTTP requests
# TYPE http_requests_total counter
http_requests_total{method="POST",endpoint="/runs"} 4532
datapipe_active_runs 3
datapipe_queue_size 12
```

■ Format text/plain Prometheus. Protéger avec IP whitelist en production.

■ Backend : Utiliser la lib `prometheus-fastapi-instrumentator`.

GET /api/v1/system/version

Version de l'API

Retourne la version déployée.

Exemple de réponse

```
{ "version": "1.0.0", "build": "abc123", "deployed_at": "2026-06-01" }
```

GET /api/v1/system/limits

Quotas et limites

Endpoints Avancés — Interface n8n+

8 endpoints complémentaires pour une expérience n8n complète

GET

/api/v1/pipelines/{pipeline_id}/nodes/{node_id}/test-data

Données de test d'un nœud

Retourne les données produites par ce nœud lors du dernier run réussi — sans relancer.

Exemple de réponse

```
{ "source": "last_run", "run_id": "run_04J...", "run_date": "2026-06-01",
  "rows": [{"Dakar", 4250000, 87}, ...], "columns": ["region", "total", "count"],
  "total_rows": 298 }
```

■ Frontend : Afficher dans la Console quand on clique sur un nœud déjà exécuté. Évite de re-run le pipeline entier.

POST

/api/v1/transform/mock-data/generate

Générer des données fictives réalistes

Produit un jeu de données factice adapté au domaine métier. Utile pour tester un pipeline sans vraies données.

Corps de la requête (JSON)

Champ	Type		Description
schema	array	● req	Schéma cible : [{ "name": "montant", "type": "FLOAT", "range": [1000, 100000] }]
rows	int	● req	Nombre de lignes à générer (max 10000)
domain	string	■ opt	"banking" "logistics" "healthcare" "retail" — adapte les valeurs
locale	string	■ opt	"fr_CI" "fr_SN" "fr_CM" — adapte les noms et lieux

Exemple de réponse

```
{ "file_id": "file_mock_01J...",
  "rows_generated": 1000,
  "sample": [
    { "client_id": 42, "montant": 75400.0, "region": "Abidjan", "date": "2026-03-15" }
  ] }
```

■ Frontend : Bouton "Générer données de test" dans le nœud CSV Import quand aucun fichier n'est uploadé.

■ Backend : Utiliser Faker avec locale adaptée. Stocker comme fichier temporaire avec TTL 24h.

GET

/api/v1/pipelines/{pipeline_id}/diff

Diff entre deux versions

Retourne les différences entre deux versions d'un pipeline (nœuds ajoutés, supprimés, modifiés).

Paramètres URL / Query

Champ	Type		Description
version_a	string	● req	ID de la version de référence
version_b	string	● req	ID de la version à comparer

Exemple de réponse

```
{ "added_nodes": [ { "id": "node_6", "type": "ai_transform" } ],
  "removed_nodes": [],
```

```
"modified_nodes": [{ "id": "node_2", "field": "config.value", "from": "30000", "to": "50000" }],
"added_edges": [ { "source": "node_5", "target": "node_6" } ],
"removed_edges": [ ] }
```

■ **Frontend** : Afficher dans le panneau *Versions*. Mettre en surbrillance les nœuds modifiés sur un canvas read-only.

POST

/api/v1/pipelines/{pipeline_id}/nodes/{node_id}/pin-data

Épingler les données d'un nœud

Sauvegarde le résultat d'un nœud pour le réutiliser dans les prochains runs sans ré-exécuter. Feature phare n8n.

Corps de la requête (JSON)

Champ	Type		Description
run_id	string	● req	ID du run dont épingler le résultat

Exemple de réponse

```
{ "pinned": true, "rows_pinned": 342, "pinned_at": "2026-06-01T14:00:00Z" }
```

■ Les nœuds en amont du nœud épinglé sont court-circuités lors des prochains runs.

■ **Frontend** : Icône "épingle" sur chaque nœud exécuté. Badge visuel distinctif sur les nœuds avec données épinglées.

■ **Backend** : Stocker le DataFrame sérialisé en Parquet. Référencer dans `pipeline_node_pins`.

GET

/api/v1/pipelines/{pipeline_id}/nodes/{node_id}/pinned-data

Lire les données épinglées

Retourne les données épinglées d'un nœud.

Exemple de réponse

```
{ "pinned": true, "pinned_at": "2026-06-01",
"rows": 342, "columns": ["montant", "region"],
"sample": [...] }
```

DELETE

/api/v1/pipelines/{pipeline_id}/nodes/{node_id}/pinned-data

Supprimer les données épinglées

Retire l'épingle — le nœud sera ré-exécuté normalement au prochain run.

Exemple de réponse

```
{ "pinned": false }
```

POST

/api/v1/pipelines/merge

Fusionner deux pipelines

Combine les nœuds de deux pipelines en un seul. Utile pour assembler des briques réutilisables.

Corps de la requête (JSON)

Champ	Type		Description
source_pipeline_id	string	● req	Pipeline à fusionner dans le cible
target_pipeline_id	string	● req	Pipeline cible qui reçoit les nœuds
offset_x	int	■ opt	Décalage X pour éviter le chevauchement des nœuds
offset_y	int	■ opt	Décalage Y

Exemple de réponse


```
{ "pipeline_id": "pip_target...", "nodes_added": 4, "edges_added": 2 }
```

GET**/api/v1/marketplace/nodes****Catalogue de nœuds communautaires**

Retourne les nœuds personnalisés publiés par la communauté (comme le marketplace n8n).

Paramètres URL / Query

Champ	Type		Description
category	string	■ opt	"finance" "logistics" "ai" "africa"
search	string	■ opt	Recherche par nom
sort	string	■ opt	"popular" "recent" "rating"

Exemple de réponse

```
{ "nodes": [  
  { "id": "mkt_01J...", "name": "OHADA Compliance Check",  
    "author": "dev_cameroun", "downloads": 1240,  
    "rating": 4.8, "category": "finance",  
    "description": "Vérifie la conformité des données comptables OHADA" }  
]
```

■ Feature prévue pour v2. Documenter maintenant pour ne pas casser l'API plus tard.

■ Frontend : Onglet "Marketplace" dans le NodePanel de gauche. Bouton "Installer" par nœud.

Codes d'erreur & Conventions

Format standard des erreurs et règles globales

Format d'erreur standard — toutes les erreurs respectent ce format :

```
{ "error": {  
  "code": "PIPELINE_NOT_FOUND", // code machine  
  "message": "Pipeline pip_xyz not found", // message lisible  
  "status": 404, // HTTP status  
  "details": { "pipeline_id": "pip_xyz" } // contexte additionnel  
} }
```

Code HTTP	Code machine	Cas d'usage
400	VALIDATION_ERROR	Corps de requête invalide (champ manquant, mauvais type)
401	UNAUTHORIZED	Token manquant, expiré ou invalide
403	FORBIDDEN	Authentifié mais rôle insuffisant pour cette action
404	NOT_FOUND	Ressource inexistante ou supprimée
409	CONFLICT	Conflit : slug déjà pris, cycle détecté
422	UNPROCESSABLE	Données valides mais logiquement incohérentes
429	RATE_LIMITED	Trop de requêtes — header Retry-After inclus
500	INTERNAL_ERROR	Erreur serveur non gérée — à logger immédiatement
503	SERVICE_UNAVAILABLE	Service en maintenance ou surchargé

Conventions globales

IDs	Format ULID : "pip_01J..." — trié chronologiquement, URL-safe
Dates	ISO 8601 UTC : "2026-06-01T14:00:00Z" — toujours en UTC
Pagination	{ "data": [...], "pagination": { "total", "page", "per_page" } }
Versioning	URL-based : /api/v1/... — pas de breaking change sans bump de version
Auth	Authorization: Bearer — header obligatoire sauf /health et /shared/*
Rate limits	100 req/min par IP pour les endpoints publics. 1000 req/min par org pour les endpoints auth.
CORS	Origines whitelist configurables par workspace. Wildcard * désactivé en production.

